



University of Jordan

School of Engineering and Technology

Department of Computer Engineering

Practical Numerical Analysis (CPE313)

Experiment 1 - MATLAB Fundamentals I

Author: Dr. Ashraf E. Suyyagh

Table of Contents

Experiment 1 - MATLAB Fundamentals I	1
What is MATLAB?	1
MATLAB Environment	2
MATLAB as a Numerical Calculator	3
Order of Precedence of Arithmetic Operations.....	4
MATLAB Variables and the Assignment Operator	5
Special Variables and Constants.....	6
Working with Complex Numbers	7
Complex Number Functions	10
MATLAB Built-in Mathematical Functions	11
Numeric Display Formats.....	14
Discrete Mathematics in MATLAB	16
Sets in MATLAB.....	16
Discrete Mathematics	17
Performance Timing in MATLAB	19
Number Rounding and Rational Fraction Approximation	19
Commands for Managing the Work Session	21
MATLAB Help and Documentation.....	24

What is MATLAB?

MathWorks has developed and maintained MATLAB since the 1970s. MATLAB is a programming language and software environment that manages data interactively.

MATLAB is historically oriented towards:

- Matrix operations and linear algebra.
- Numerical analysis.
- Fast and easy data plotting
- Interface with other programming languages
- Symbolic processing (e.g. differentiation and integration with symbols as you would solve problems in Calculus).

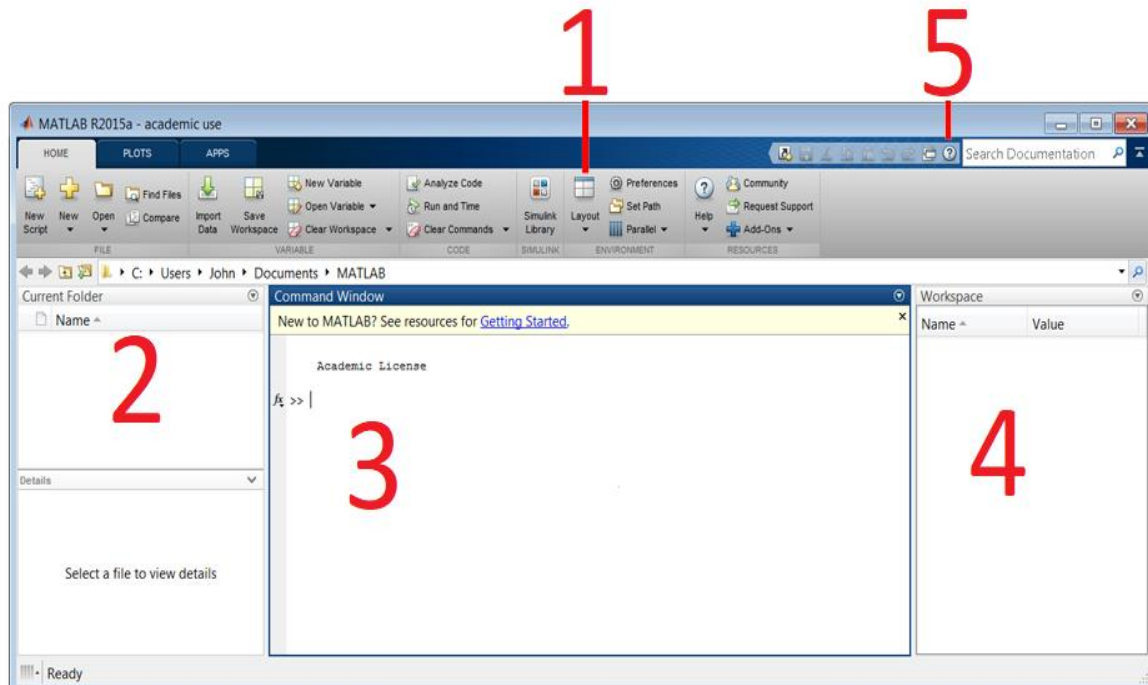
MATLAB has a wide array of toolboxes for image processing, signal processing, statistics, machine learning, control and automotive systems among dozen others (may require additional paid licenses, so we might not have all of them in the lab).

MATLAB provides through an additional integrated product called Simulink graphical model-based design and simulation for numerous engineering, scientific, and economic systems (may require additional paid licenses, so we might not have all of them in the lab).

C++ and Java are **compiled languages** where all the code is first compiled, then an executable is generated and run afterwards. In contrast, MATLAB is an **interpreted language**. **There is no code compilation. MATLAB executes commands in order during run-time.** That is; it understands (interprets) the commands one-by-one as they appear in the code/script.

MATLAB Environment

The main screen of MATLAB has five main windows:



1. The **Ribbon tab** where you have access to the IDE operations: opening new or existing files, save files, import data from files (e.g. Excel or CSV), set working path, run your code, and access MATLAB plots and numerous toolboxes.
2. **Current working directory** where you save all your MATLAB scripts, functions, plots ... etc. MATLAB commands expect to work with files in the current working directory (**More on this later**). You will learn how to setup the working path later.
3. **Command Line** where you write and execute your MATLAB commands. You will see the output of your commands in the same window.
4. **Workspace** where you all your session variables are saved and accessed.
5. **Help and Documentation** where you can search for any topic or command.

MATLAB as a Numerical Calculator

You can use MATLAB to quickly find the output of many mathematical operations. For example, try to copy/paste these codes one by one in the **command window**, then press **Enter** to see the output after each one.

```
10 + 10
```

```
ans = 20
```

```
65 - 35
```

```
ans = 30
```

```
85 * 5
```

```
ans = 425
```

```
2 ^ 10
```

```
ans = 1024
```

```
8 / 4
```

```
ans = 2
```

```
8 \ 4
```

```
ans = 0.5000
```

Note that in the last operation, we used the backslash instead of the division operator. The backslash acts as a division operator but in the reverse order. So, $8 \setminus 4$ is equivalent to $4 / 8 = 0.5$. This is called **backward division** while the normal division case is called **forward division**.

Notice that after each operation that you execute, your answer is stored in a special variable called **ans** (that is short for answer). There is only one **ans** variable that is constantly changing as it always stores the result of the most recent operation. Each subsequent operation essentially overrides the previous answer with its own result and stores it in **ans**. You can see the **ans** variable to your right in the **Workspace** window. The workspace window displays all variables used in your session. Later on, when you work with large data (e.g. arrays), you can double-click on the variable and see its content in an Excel-like sheet.

MATLAB saves all your previous keystrokes. While in the command window, press the **Up-Arrow** on your keyboard. You will see all your previous commands. You can also access all previous commands by typing **commandhistory**. From there, you can select any command or group of commands by pressing (**SHIFT + LEFT Mouse button**) on each of them. You can reevaluate these commands by (**Right-Click on your selection --> Evaluate Selection**). You can edit the commands too by using **Backspace** and **Delete** keys and **Left** and **Right** arrow keys.

Order of Precedence of Arithmetic Operations

One easy way to remember the rules of precedence of arithmetic operations is by remembering the word: **PEMDAS**. Each letter stands for the first letter of the operations in their order of precedence.

1. **P**arenthesis
2. **E**xponentiation
3. **M**ultiplication and **D**ivision
4. **A**ddition and **S**ubtraction

When the precedence level is equal, evaluation is performed from **LEFT** to **RIGHT**.

Calculate the output of these examples by hand. Compare your answer by executing these examples in the command window. Copy/paste these expressions one by one in the **command window**, then press **Enter** to see the output after each one.

```
8 + 3 * 5
```

```
ans = 23
```

```
8 + (3 * 5)
```

```
ans = 23
```

```
(8 + 3) * 5
```

```
ans = 55
```

```
4 ^ 2 - 12 - 8 / 2 * 2
```

```
ans = -4
```

```
4 ^ 2 - 12 - 8 / (2 * 2)
```

```
ans = 2
```

```
4 ^ 2 - 12 - 8 \ (2 * 2)
```

```
ans = 3.5000
```

```
3 * 4 ^ 2 + 5
```

```
ans = 53
```

```
(3 * 4) ^ 2 + 5
```

```
ans = 149
```

```
27 ^ ( 1 / 3 ) + 32 ^ (0.2)
```

```
ans = 5
```

```
27 ^ ( 1 / 3 ) + 32 ^ 0.2
```

```
ans = 5
```

```
27 ^ 1 / 3 + 32 ^ 0.2
```

```
ans = 11
```

MATLAB Variables and the Assignment Operator

Unlike other programming languages like C++ or Java, you **DON'T** need to declare a MATLAB variable and define a type prior to its use. You can directly assign values to variables in MATLAB.

By default, and unless otherwise defined, **ALL** numeric values are treated as **double-precision floating-point** values in MATLAB.

Variables can directly store text, tables, structures, vectors and arrays (matrices) among other data types (more on this later).

As explained before, if no variable is used to assign the result to it, the default variable where the answer is stored is **ans**.

To define a new variable, simply write the variable name followed by the assignment operator (=) and the value to be stored in the variable. **ONLY ONE** variable can be written to the left hand side of the assignment operator. Expressions such as $x + 5 = 20$ are **NOT ALLOWED**.

Copy/paste these examples one by one in the **command window**, then press **Enter** to see the output after each one.

```
a = 25
```

```
a = 25
```

```
b = a + 25
```

```
b = 50
```

```
5 * 20
```

```
ans = 100
```

```
c = ans * 10
```

```
c = 1000
```

```
d = 8
```

```
d = 8
```

```
d = 'numerical'
```

```
d = 'numerical'
```

Notice that we did not define any datatypes for our variables in the above examples (**There is no *int*, or *float*, or *double*, or *string***).

Note that the output of $5 * 20$ is implicitly stored in the **ans** variable (since the output was not assigned (stored) into any other variable). Also note that we can use the **ans** variable in further computations ($c = \text{ans} * 10$) even though it is not a recommended practice because **ans** always changes value.

Interestingly, in the above example, the variable **d** initially holds the numerical value 8. Then, we override it to store the **string** 'numerical'. This is permissible since in MATLAB, if the variable already exists, MATLAB changes its contents and, if necessary, allocates new storage.

Special Variables and Constants

MATLAB has a set of special variables (case-sensitive):

- **ans**: Temporary variable containing the most recent answer.
- **i** and **j**: the imaginary unit $\sqrt{-1}$.
- **inf** or **Inf**: the infinity ∞ .
- **NaN**: stands for **Not an Number**; indicates an undefined numerical result.
- **pi**: the math number $\pi = 3.141592\dots$
- **eps**: by default, this variable represents the machine's accuracy in double precision. It represents the distance from 1.0 to the next largest double-precision number. On a typical machine running a modern Intel processor $\epsilon = 2.220446049250313\text{e-}16$
- **realmax**: Largest positive floating-point number
- **realmin**: Smallest normalized floating-point number

Copy/paste these examples one by one in the **command window**, then press **Enter** to see the output after each one:

```
d = 1 / 0
```

```
d = Inf
```

```
e = 0 / 0
```

```
e = NaN
```

While the above operations will result in an error on a hand-held calculator or in some other programming languages, MATLAB can handle results of infinity or undefined values.

Remember that in other programming languages such as C++ and Java that the range of signed integers of type **int** is -2147483648 to +2147483647. Now, try storing this number in MATLAB:

```
g = 4567892345638
```

```
g = 4.5679e+12
```

Surprisingly, the variable **g** was indeed declared, reserved in memory, and stored the number 4567892345638. But how could it store an integer value that exceeds the possible range?

Use the **class** command to determine the data type of the variable **g**

```
class(g)
```

```
ans = 'double'
```

Remember that MATLAB stores **all numeric values by default** as double-precision floating point numbers. This enables MATLAB to handle much larger range of integers and floating-point numbers. This becomes clear knowing that integers are a subset of floating-point numbers.

The largest/smallest integer number MATLAB can store **precisely** and **exactly in the default double floating-point type variable** without any errors is $2^{53} = \pm 9,007,199,254,740,992$. All other integers stored in MATLAB above or below this number are not exact integers. You should review the IEEE 754 standard which you have studied in the Computer Organization and Design course to know why.

Working with Complex Numbers

We have already introduced that the letters **i** and **j** are predefined variables in MATLAB to represent the imaginary number $\sqrt{-1}$. Note that you don't need a multiplication sign between the number and the imaginary unit **i** or **j**. Copy/paste these examples one-by-one in the **command window**, then press **Enter** to see the output after each one.

```
a = 1 + 1j
```

```
a = 1.0000 + 1.0000i
```

```
b = 5 + 6i
```

```
b = 5.0000 + 6.0000i
```

```
c = a + b
```

```
c = 6.0000 + 7.0000i
```

When you are going to use **i** or **j** with any constant to represent a complex number, you must write the imaginary value without a multiplication sign. Even if the value is **1**, it is **recommended** to explicitly precede it with **1**, so it looks like 1i or 1j.

The above recommendation is important. Sometimes, you might by mistake override the variables **i** and **j** by any other value and MATLAB allows that without issues!

```
i = 2
```

```
i = 2
```

```
j = 3
```

```
j = 3
```

To restore the values of **i** and **j** to the default $\sqrt{-1}$, simply delete the variables by using the **clear** command and MATLAB will revert to the original meaning.

```
clear i j
```

As long as you don't override the values of **i** and **j**, these numbers are all complex.

```
clear i j
a1 = 1 + i      % COMPLEX
```

```
a1 = 1.0000 + 1.0000i
```

```
b1 = 1 + 1i     % COMPLEX
```

```
b1 = 1.0000 + 1.0000i
```

```
c1 = 1 + 1 * i  % COMPLEX
```

```
c1 = 1.0000 + 1.0000i
```

```
d1 = 1 + 2i     % COMPLEX
```

```
d1 = 1.0000 + 2.0000i
```

```
e1 = 1 + 2 * i  % COMPLEX
```

```
e1 = 1.0000 + 2.0000i
```

```
f1 = 5 + 6j     % COMPLEX
```

```
f1 = 5.0000 + 6.0000i
```

```
g1 = 5 + 6*j    % COMPLEX
```

```
g1 = 5.0000 + 6.0000i
```

However, once you override the default imaginary numbers with other values, the same expressions will be interpreted differently. Consider:

```
i = 1          % Overriding i and j
```

```
i = 1
```

```
j = 3
```

```
j = 3
```

```
a2 = 1 + i     % NOT COMPLEX
```

```
a2 = 2
```



```
b2 = 1 + 1i      % COMPLEX
```

```
b2 = 1.0000 + 1.0000i
```

```
c2 = 1 + 1 * i   % NOT COMPLEX
```

```
c2 = 2
```

```
d2 = 1 + 2i      % COMPLEX
```

```
d2 = 1.0000 + 2.0000i
```

```
e2 = 1 + 2 * i   % NOT COMPLEX
```

```
e2 = 3
```

```
f2 = 5 + 6j      % COMPLEX
```

```
f2 = 5.0000 + 6.0000i
```

```
g2 = 5 + 6*j     % NOT COMPLEX
```

```
g2 = 23
```

Notice that whenever you place a multiplication sign between the imaginary variable **i** or **j** and its coefficient, MATLAB no longer interpret **i** and **j** as $\sqrt{-1}$. Instead, they are considered as real-numbered variables and MATLAB uses the new overridden values. Also, if the imaginary coefficient is 1 and you forget to explicitly write the imaginary part as **1i** or **1j**, then in this case, the overridden value of **i** or **j** will be used.

The order of precedence will yield different values if you are not too careful in writing your expressions. Consider:

```
clear i j      % ensure default imaginary values are in use
```

```
a = 9/2*i      % a = 4.5i
```

```
a = 0.0000 + 4.5000i
```

```
b = 9/2i       % b = -4.5i
```

```
b = 0.0000 - 4.5000i
```

MATLAB treats the first expression as having three terms and will evaluate the expression according to the rules of precedence. Since division and multiplication have equal precedence, the expression will have the same result as:

```
a = (9/2)*i    % a = 4.5i
```

```
a = 0.0000 + 4.5000i
```

Yet, MATLAB treats the second expression as having two terms only (due to the absence of the multiplication sign), so the second expression is basically the same as:

```
b = 9 / (2i) % b = -4.5i
```

```
b = 0.0000 - 4.5000i
```

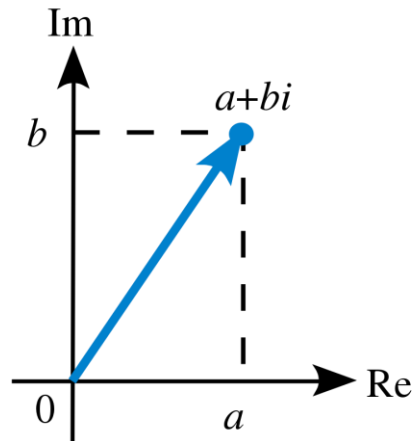
Perhaps a direct way to create MATLAB variable is by using MATLAB complex built-in function. If we represent a complex number as $a + bi$, then we can create a complex number by writing **complex(a, b)**

```
complex(5, 10) % This is equivalent to 5 + 10i
```

```
ans = 5.0000 + 10.0000i
```

Complex Number Functions

While working with complex numbers, there are five main functions that we might frequently need. Splitting the imaginary and real parts. Determining the complex conjugate, and most importantly computing the magnitude and angle of the complex vector. Remember that a complex number $a + bi$ can be projected on the complex plane as:



MATLAB has all these operations as built-in functions. The following example illustrates how to use them:

```
a = 5+3j
```

```
a = 5.0000 + 3.0000i
```

```
b = complex(7, -8)
```

```
b = 7.0000 - 8.0000i
```

```
abs(a) % Magnitude of a complex number
```

```
ans = 5.8310
```

```
abs(b)
```

```
ans = 10.6301
```

```
angle(a)    % Phase angle of complex number (in RADIANS)
```

```
ans = 0.5404
```

```
angle(b)
```

```
ans = -0.8520
```

```
conj(a)     % Complex Conjugate
```

```
ans = 5.0000 - 3.0000i
```

```
conj(b)
```

```
ans = 7.0000 + 8.0000i
```

```
real(a)     % Extract real part of complex number
```

```
ans = 5
```

```
real(b)
```

```
ans = 7
```

```
imag(a)     % Extract imaginary part of complex number
```

```
ans = 3
```

```
imag(b)
```

```
ans = -8
```

MATLAB Built-in Mathematical Functions

MATLAB supports lots of common mathematical functions. We provide a listing of the major ones which you already know from algebra and calculus courses.

Please note that the **TRIGONOMETRIC FUNCTIONS** IN MATLAB USE **RADIAN** MEASURE by default.

Mathematical Functions

Exponential and Logarithmic Functions	
$\exp(x)$	Exponential; e^x .
$\log(x)$	Natural logarithm; $\ln(x)$.
$\log_{10}(x)$	Common (base 10) logarithm; $\log(x) = \log_{10}(x)$.
$\text{sqrt}(x)$	Square root; $\sqrt{x}$.

Trigonometric Functions	
$\text{acos}(x)$	Inverse cosine; $\arccos x = \cos^{-1}(x)$.
$\text{acot}(x)$	Inverse cotangent; $\text{arccot } x = \cot^{-1}(x)$.
$\text{acsc}(x)$	Inverse cosecant; $\arcs x = \csc^{-1}(x)$.
$\text{asec}(x)$	Inverse secant; $\text{arcsec } x = \sec^{-1}(x)$.
$\text{asin}(x)$	Inverse sine; $\arcsin x = \sin^{-1}(x)$.
$\text{atan}(x)$	Inverse tangent; $\text{arctan } x = \tan^{-1}(x)$.
$\text{atan2}(y, x)$	Four-quadrant inverse tangent.
$\cos(x)$	Cosine; $\cos(x)$.
$\cot(x)$	Cotangent; $\cot(x)$.
$\csc(x)$	Cosecant; $\csc(x)$.
$\sec(x)$	Secant; $\sec(x)$.
$\sin(x)$	Sine; $\sin(x)$.
$\tan(x)$	Tangent; $\tan(x)$.

Hyperbolic Functions	
$\text{acosh}(x)$	Inverse hyperbolic cosine; $\cosh^{-1}(x)$.
$\text{acoth}(x)$	Inverse hyperbolic cotangent; $\coth^{-1}(x)$.
$\text{acsch}(x)$	Inverse hyperbolic cosecant; $\text{csch}^{-1}(x)$.
$\text{asech}(x)$	Inverse hyperbolic secant; $\text{sech}^{-1}(x)$.
$\text{asinh}(x)$	Inverse hyperbolic sine; $\sinh^{-1}(x)$.
$\text{atanh}(x)$	Inverse hyperbolic tangent; $\tanh^{-1}(x)$.
$\cosh(x)$	Hyperbolic cosine; $\cosh(x)$.
$\coth(x)$	Hyperbolic cotangent; $\cosh(x)/\sinh(x)$.
$\text{csch}(x)$	Hyperbolic cosecant; $1/\sinh(x)$.
$\text{sech}(x)$	Hyperbolic secant; $1/\cosh(x)$.
$\sinh(x)$	Hyperbolic sine; $\sinh(x)$.
$\tanh(x)$	Hyperbolic tangent; $\sinh(x)/\cosh(x)$.

Let's try some of these functions:

```
a = sqrt(9)
```

```
a = 3
```

```
b = sqrt(-16)      % returns a complex number 0 + 4i
```

```
b = 0.0000 + 4.0000i
```

Since $10^2 = 100$, then $\log_{10}(100) = 2$

```
c = log10(100)
```

```
c = 2
```

and $\ln(e^3) = 3$

```
d = log(exp(3))
```

```
d = 3
```

But what if we need to find the natural logarithm for a base different than 10 or e ? In this case, we need to use mathematical rules:

$$\log_b(a) = \frac{\log_{10}(a)}{\log_{10}(b)}$$

so if you need to compute $\log_3(6) = \frac{\log_{10}(6)}{\log_{10}(3)}$ which can be easily computed in MATLAB as:

```
log10(6) / log10(3)
```

```
ans = 1.6309
```

As we know, $\sin(30^\circ) = 0.5$, attempting this directly in MATLAB will yield the wrong result:

```
sin(30)
```

```
ans = -0.9880
```

Remember, trigonometric functions in MATLAB use **RADIAN** as function input by default, so you must convert the value from decimal to RADIANS by multiplying it by $\frac{\pi}{180}$:

```
sin(30*pi/180)
```

```
ans = 0.5000
```

Similarly, we know that $\cos^{-1}(0.5) = 60^\circ$, attempting this directly in MATLAB will yield the result in radian:

```
acos(0.5)
```

```
ans = 1.0472
```

so don't forget to correct the output - if necessary - by converting it to decimal by multiplying it by $\frac{180}{\pi}$.

```
acos(0.5)*180/pi
```

```
ans = 60.0000
```

If you wish to have the square of a trigonometric function, say $\cos^2(x)$, then the correct way to do it is to square the whole function in MATLAB, as in $(\cos(x))^2$

Finally, we already learnt that using the `abs()` function with complex numbers returns the magnitude of the complex numbers. You can use the `abs()` function with real numbers as well, in this case, it means the absolute value $|x|$, try:

```
abs(-5)
```

```
ans = 5
```

Numeric Display Formats

MATLAB's **format** command gives us control on HOW TO DISPLAY the output numbers. They **DO NOT** change the actual accuracy of the number stored in memory. Just what you see on the screen. Some formatting commands affect the look and feel of the workspace (e.g. line spacing). The following table lists all available output formatting commands:

Numeric Display Formats	
<code>format short</code>	Four decimal digits (default).
<code>format long</code>	16 decimal digits.
<code>format short e</code>	Five digits plus exponent.
<code>format long e</code>	16 digits plus exponents.
<code>format bank</code>	Two decimal digits.
<code>format +</code>	Positive, negative, or zero.
<code>format rat</code>	Rational approximation.
<code>format compact</code>	Suppresses some line feeds.
<code>format loose</code>	Resets to less compact display mode.

The format **compact** and **loose** functions control the **line spacing** in the command window.

Let's try few of them with the number π

```
format short  
pi
```

```
ans = 3.1416
```

```
format long  
pi
```

```
ans =  
3.141592653589793
```

```
format short e  
pi
```

```
ans =  
3.1416e+00
```

```
format long e  
pi
```

```
ans =  
3.141592653589793e+00
```

```
format bank  
pi
```

```
ans =  
3.14
```

```
format +  
pi
```

```
ans =  
+
```

```
format rat  
pi
```

```
ans =  
355/113
```

```
format hex      % Hexadecimal representation  
pi
```

```
ans =  
400921fb54442d18
```

```
format          % Resets to the default formatting
```

Discrete Mathematics in MATLAB

In the previous experiment, we learnt how to do various vector and matrix operations such as logical, arithmetic, and other manipulations. We mostly applied basic trigonometric and linear algebra math. In this section, we will learn how to handle mathematical sets in the same way you learnt in basic math and discrete math courses.

Sets in MATLAB

MATLAB allows users to enter mathematical sets as a vector or a matrix (array) - more on vectors and matrices later. For example, let's generate the two mathematical sets S_1 and S_2 :

$$S_1 = \{1, 5, 4, 6, 7, 7, 3, 5, 2, 8, 0, 1, 1, 2, 7, 8, 5, 4, 2, 4, 1, 4\}$$

$$S_2 = \{0, 9, 8, 9, 9, 7, 3, 10, 5, 3, 6, 2, 4, 1, 1, 1, 11, 3\}$$

as two vectors:

```
S1 = [1, 5, 4, 6, 7, 7, 3, 5, 2, 8, 0, 1, 1, 2, 7, 8, 5, 4, 2, 4, 1, 4];
S2 = [0, 9, 8, 9, 9, 7, 3, 10, 5, 3, 6, 2, 4, 1, 1, 1, 11, 3];
```

To extract the numbers that occur in the set without any repetitions; that is to extract the unique numbers in the set, use the function **unique**:

```
unique(S1)
```

```
ans = 1×9
      0      1      2      3      4      5      6      7      8
```

```
unique(S2)
```

```
ans = 1×12
      0      1      2      3      4      5      6      7      8      9     10     11
```

The function **unique** returns the existing elements in the set in sorted ascending order. If you wish to see the unique numbers in the set in the order they appear in, then use:

```
unique(S1, 'stable')
```

```
ans = 1×9
      1      5      4      6      7      3      2      8      0
```

To merge two sets together; that is, to perform the operation $S_1 \cup S_2$, use the **union** command:

```
union(S1, S2)
```

```
ans = 1×12
      0      1      2      3      4      5      6      7      8      9     10     11
```


The union commands can work with sets stored in matrices of different sizes and shapes. There is no need for the vectors or matrices to match each other's dimensions.

To find the elements that appear in both sets; that is, to perform the operation $S_1 \cap S_2$, use the **intersect** command:

```
intersect(S1, S2)
```

```
ans = 1×9
      0      1      2      3      4      5      6      7      8
```

The default output is sorted in ascending order, unless you use 'stable' as before.

To see the elements that exist in one set but not the other, use set difference command **setdiff**. Note that the order you use in performing this operation matters:

```
setdiff(S1, S2)      % All elements in S1 appear in S2, so set difference is
empty
```

```
ans =
      1×0 empty double row vector
```

```
setdiff(S2, S1)
```

```
ans = 1×3
      9     10     11
```

Another method to check if an element exists in a set is to use the **ismember** command. It returns **logical** 0 if the number does not exist in the set, and 1 otherwise. To check if the numbers **5, 8, 10** and **15** exist in S_1 :

```
ismember([5, 8, 10, 15], S1)
```

```
ans = 1×4 logical array
      1      1      0      0
```

Discrete Mathematics

MATLAB has extremely powerful functions in the domain of discrete mathematics. We list them in the following table:

factor	Returns the prime factors of the number n
isprime	Determine which array elements are prime
primes	Lists prime numbers less than or equal to the input value
nextprime	Returns the next prime after input number n
prevprime	Returns the previous prime before input number n
nthprime	Returns the n^{th} prime number
gcd	Greatest common divisor
lcm	Least common multiple
factorial	Factorial of input $n!$
perms	Returns a matrix containing all permutations of the elements of the input vector in reverse lexicographic order

Let's try few examples. To list all prime numbers under 50, write:

```
primes(50)
```

```
ans = 1×15
      2      3      5      7      11      13      17      19      23      29      31      37
      41 ...
```

The 100th prime number is:

```
nthprime(100)
```

```
ans = 541
```

To factor the number 567838 into its factors, use:

```
factor(567840)
```

```
ans = 1×10
      2      2      2      2      2      3      5      7      13      13
```

The least common multiple of 10, 6 is:

```
lcm(10, 6)
```

```
ans = 30
```

To list all possible combinations of the numbers 3, 7, 9, use the function **perms**:

```
perms([3, 7, 9])
```

```
ans = 6×3
     9     7     3
     9     3     7
     7     9     3
     7     3     9
     3     9     7
     3     7     9
```

Performance Timing in MATLAB

Many times, we are not only interested in writing a correct code, but also an efficient code. Performance timing is important in order to compare how much time has lapsed in writing two versions of the same function or algorithm.

There are many ways to profile the timing of your code. but we will only explain a simple one:

- Use the **tic** and **toc** commands at the beginning and end of the function you want to profile. Once you execute the **toc** command, the elapsed time will appear. (**RECOMMENDED**)

```
tic
% Your algorithm goes here, do something
toc
```

Elapsed time is 0.001809 seconds.

NOTE: Usually, we do not profile the execution time based on a single execution. We place our function or algorithm in a loop and run it hundreds of times, then we take the average of execution times.

We shall learn about more MATLAB tools for code timing and profiling in the next experiment.

Number Rounding and Rational Fraction Approximation

MATLAB also offers functions that perform the rounding operations that you are familiar with from math or previous programming courses. We summarize these functions in the following table:

Numeric Functions	
<code>ceil</code>	Rounds to the nearest integer toward ∞ .
<code>fix</code>	Rounds to the nearest integer toward zero.
<code>floor</code>	Rounds to the nearest integer toward $-\infty$.
<code>round</code>	Rounds towards the nearest integer.
<code>sign</code>	Signum function.

The **ceil** and **floor** functions are the ones with the math symbols $\lceil x \rceil$ and $\lfloor x \rfloor$, respectively. The **fix** function returns the number closest to zero. The differences are clearer when comparing the results of positive and negative numbers. The **sign** function is a very useful function which returns the signs of the input number(s). We show some insightful examples in the following figure:

ceil (-5.27)	-5
fix (-5.27)	-5
floor (-5.27)	-6
round (-5.27)	-5
round (-5.27, 1)	-5.3
sign	-1

ceil (5.27)	6
fix (5.27)	5
floor (5.27)	5
round (5.27)	5
round (5.27, 1)	5.3
sign (5.27)	1



ceil (-1.67)	-1
fix (-1.67)	-1
floor (-1.67)	-2
round (-1.67)	-2
round (-1.67, 1)	-1.7
sign (-1.67)	-1

ceil (1.67)	2
fix (1.67)	1
floor (1.67)	1
round (1.67)	2
round (1.67, 1)	1.7
sign (1.67)	1

The rounding function **round** has different syntax for different cases; the default syntax takes one input (the number to be rounded) and works at the “half digit” boundary. **Positive numbers** with fractional parts 0.5 - 0.99... get rounded to the next largest number; otherwise, they get rounded to the lower integer value. Negative numbers rounding works in reverse. Consult the examples in the table above.

Let us try few rounding examples; let's round the number: 67585.891:

```
round(67585.891)           % Default roundig behaviour
```

```
ans = 67586
```

If we want to round to one decimal point:

```
round(67585.891, 1)
```

```
ans = 6.7586e+04
```

Notice that the result is given in engineering format, and that we cannot see the effect clearly. Precede the **round** command with **format long**

```
format long
round(67585.891, 1)
```

```
ans =
6.758589999999999e+04
```

Notice that we expect the result to be 67585.9 yet MATLAB returns a result of 67585.899999999..... This is due to the IEEE 754 standard and hardware design of floating-point units inside computers

which does not always allow us to represent floating-point error precisely. We shall discuss this in detail in a future experiment.

and to two decimal points:

```
format long
round(67585.891, 2)

ans =
    6.758589000000000e+04
```

Notice that we expect the result to be 67585.89 and MATLAB indeed returns the correct result.

Alternatively, one can round to the nearest 10 or nearest 100 or 1000 by writing:

```
round(67585.891, -1)

ans =
    67590
```

```
round(67585.891, -2)

ans =
    67600
```

```
round(67589.891, -3)

ans =
    68000
```

Commands for Managing the Work Session

In this section, we will present commands that are quite useful in managing your workspace.

We already know that all variables you declare in any MATLAB session can be found under the **Workspace** window (default location --> left pane). You can also list the session variables in the command window by using this command:

```
who

Your variables are:

S1  a    a2  b    b2  c1  d    d2  e1  f1  g    g2
S2  a1  ans b1  c    c2  d1  e    e2  f2  g1
```

If you need further details about your variables, you can use:

whos

Name	Size	Bytes	Class	Attributes
S1	1x22	176	double	
S2	1x18	144	double	
a	1x1	8	double	
a1	1x1	16	double	complex
a2	1x1	8	double	
ans	1x1	8	double	
b	1x1	16	double	complex
b1	1x1	16	double	complex
b2	1x1	16	double	complex
c	1x1	8	double	
c1	1x1	16	double	complex
c2	1x1	8	double	
d	1x1	8	double	
d1	1x1	16	double	complex
d2	1x1	16	double	complex
e	1x1	8	double	
e1	1x1	16	double	complex
e2	1x1	8	double	
f1	1x1	16	double	complex
f2	1x1	16	double	complex
g	1x1	8	double	
g1	1x1	16	double	complex
g2	1x1	8	double	

Notice, that the size of most variables we used thus far is listed as 1x1. This is because all our variables are scalars (not vectors nor matrices (arrays), just single-valued numbers). Even the complex numbers have a size of 1x1 as they are treated as one unit. This size attribute is different than the actual size these variables take inside the memory. You can find the memory size under **Bytes**. Since most our variables are double-precision floating-point numbers, we expect they will use 8 bytes, and complex numbers will take 16 bytes to store both the real and imaginary parts. You can find the data type of your variables under the **Class attribute**.

In huge projects, you might lose track of variables. You might forget if you have used the variable 'num' for example, and you don't need to override it and cause errors in your program. To check if a variable exists, use the **exist** function. It takes in ONE variable name in single quotations or directly listed next to it. It returns 0 or 1 depending if the variable exists or not.

```
exist('num')
```

```
ans =  
0
```

```
exist a
```

```
ans =  
1
```

To delete variables from the workspace, use **clear** then list the variable names to be deleted. Let's delete variables a, b, c, and d.

```
exist a
```

```
ans =  
      1
```

```
clear a b c d  
exist a
```

```
ans =  
      0
```

To clear all variables in the workspace, simply write **clear** on its own:

```
clear
```

To clear the content of the command window, simply use:

```
clc
```

The above command does not delete the history of your commands, only the command window. You can still see all previous commands by pressing the **UP** button or writing **commandhistory**.

When you are writing large MATLAB scripts and functions (*discussed in later experiments*), you are only interested in the final output. To suppress (hide) the output of MATLAB lines, use the semicolon at the end. Compare:

```
a = (5 + 4j)*(6 - 2j)
```

```
a =  
38.000000000000000 +14.000000000000000i
```

```
b = (7 + 6j)*(7 - 9j);
```

In both cases, the output is computed and stored in the variables **a** and **b**. In the first case, MATLAB shows the output in the command window, while in the latter case, the semicolon suppressed the output from showing. To see the output of the variable **b**, you can either type the variable directly:

```
b
```

```
b =  
1.030000000000000e+02 - 2.100000000000000e+01i
```

or use the display **disp()** function:

```
disp(b)
```

```
1.030000000000000e+02 - 2.100000000000000e+01i
```

When you are writing long lines in MATLAB, and you wish to continue writing on the next line, you can use the Ellipsis (three dots ...)

```
1245 + 86844 + 767683 + 34334 + ...
456 + 97800
```

```
ans =
    988362
```

You can automatically capture and log (store) all entered command, keyboard input and command window output using the **diary** command by using it with the **on** and **off** switches. MATLAB will create a text file called *diary* in the working directory that contains all session command window interactions.

```
diary on
% Some commands
diary off
```

You can also display the content of text files inside MATLAB's command window by using the **type** command. MATLAB will look for the supplied filename inside the current path (working directory), or any added path. Otherwise, you need to provide the full path to the **type** command:

```
type sampleNumbers.txt
```

```
0.0975    0.1576    0.1419    0.6557    0.7577    0.7060    0.8235    0.4387
0.2785    0.9706    0.4218    0.0357    0.7431    0.0318    0.6948    0.3816
0.5469    0.9572    0.9157    0.8491    0.3922    0.2769    0.3171    0.7655
0.9575    0.4854    0.7922    0.9340    0.6555    0.0462    0.9502    0.7952
0.9649    0.8003    0.9595    0.6787    0.1712    0.0971    0.0344    0.1869
```

The file *sampleNumbers.txt* contains a few lines of numbers that can be printed inside the command window.

To quit MATLAB from the command line, simply write quit.

```
quit
```

MATLAB Help and Documentation

MATLAB has an extensive well-written documentation with numerous examples as well an online MATLAB answers forum. In MATLAB, there are two options to access info on all commands: a quick help command and another for full documentation. For example, if you need to quickly see the function and syntax of a MATLAB command, type **help** followed by the command name, and a short documentation will appear inside the command window and it will list all available MATLAB commands that handle binary numbers one way or another:

```
help format
```

```
format Set output format.
format with no inputs sets the output format to the default appropriate
```


for the class of the variable. For float variables, the default is **format SHORT**.

format does not affect how MATLAB computations are done. Computations on float variables, namely single or double, are done in appropriate floating point precision, no matter how those variables are displayed. Computations on integer variables are done natively in integer. Integer variables are always displayed to the appropriate number of digits for the class, for example, 3 digits to display the INT8 range -128:127. **format SHORT** and **LONG** do not affect the display of integer variables.

format may be used to switch between different output display formats of all float variables as follows:

format SHORT	Short fixed point format with 4 digits after the decimal point.
format LONG	Long fixed point format with 15 digits after the decimal point for double values and 7 digits after the decimal point for single values.
format SHORTE	Short scientific notation with 4 digits after the decimal point.
format LONGE	Long scientific notation with 15 digits after the decimal point for double values and 7 digits after the decimal point for single values.
format SHORTG	Short fixed format or scientific notation, whichever is more compact, with a total of 5 digits.
format LONGG	Long fixed format or scientific notation, whichever is more compact, with a total of 15 digits for double values and 7 digits for single values.
format SHORTENG	Engineering format with 4 digits after the decimal point and a power that is a multiple of three.
format LONGENG	Engineering format that has exactly 15 significant digits and a power that is a multiple of three.

format may be used to switch between different output display formats of all numeric variables as follows:

format HEX	Hexadecimal format.
format +	The symbols +, - and blank are printed for positive, negative and zero elements. Imaginary parts are ignored.
format BANK	Currency format with 2 digits after the decimal point.
format RATIONAL	Approximation by ratio of small integers. Numbers with a large numerator or large denominator are replaced by *.

format may be used to affect the spacing in the display of all variables as follows:

format COMPACT	Suppresses extra line-feeds.
format LOOSE	Puts the extra line-feeds back in.

If you need to access the full documentation, simply type **doc** followed by the command name. The documentation window will open outside of MATLAB.

doc format

But what if you don't know the command name? How to find a certain command that does a specific job you want? In this case, use the command **lookfor** with **one keyword** that describes what you want, and MATLAB will list all available close commands with a summary of what they do.

For example, to look for MATLAB commands that can work with binary numbers, simply write:

```
lookfor binary
```

bsxfun	- Binary Singleton Expansion Function
fread	- Read binary data from file.
fwrite	- Write binary data to file.
bin2dec	- Convert text representation of binary number to double
value	
dec2bin	- Convert decimal integer to its binary representation
binary	- Sets binary transfer type.
binary	- Sets binary transfer type.
bsxfun	- Binary Singleton Expansion Function
invokeBinaryComparison	- Invokes GE, LT etc.
bops	- o = BOPS(obj) Returns the binary operator nodes in obj
isbop	- b = ISBOP(obj) Boolean array, true if node is binary
binaryVectorToDecimal	- Convert binary vector to a decimal number.
binaryVectorToHex	- Convert binary vector to a hexadecimal character string.
binvec2dec	- Convert binary vector to decimal number.
dec2binvec	- Convert decimal number to a binary vector.
decimalToBinaryVector	- Convert decimal number to a binary vector.
hexToBinaryVector	- Convert hex number to a binary vector.
dspblkbinaryfilereader	- DSP System Toolbox binary file reader block
dspblkbinaryfilewriter	- DSP System Toolbox binary file writer block
dec2mvl	- Convert decimal integer to a binary string.
fibinscaling	- Fi Binary Point Scaling Demo
fiscalingdemo	- Perform Binary-Point Scaling
bin	- Binary representation of stored integer of fi object
isscalingbinarypoint	- Determine whether fi object has binary point scaling
BinaryPointScaling	-
bwarea	- Area of objects in binary image.
bwareafilt	- Extract objects from binary image by size.
bwareaopen	- Remove small objects from binary image.
bwboundaries	- Trace region boundaries in binary image.
bwconncomp	- Find connected components in binary image.
bwconvhull	- Generate convex hull image from binary image.
bwdist	- Distance transform of binary image.
bwdist_old	- Distance transform of binary image.
bwdistgeodesic	- Geodesic distance transform of binary image.
bweuler	- Euler number of binary image.
bwfill	- Fill background regions in binary image.
bwhitmiss	- Binary hit-miss operation.
bwlabel	- Label connected components in 2-D binary image.
bwlabeln	- Label connected components in binary image.
bwmorph	- Morphological operations on binary image.
bwmorph3	- Morphological operations on binary volume.
bwpack	- Pack binary image.
bwperim	- Find perimeter of objects in binary image.

You can see in the list two functions of interest: **bin2dec** and **dec2bin** which allows you to convert between binary and decimal numbers in text formatting. Try them out.

Experiment version 1.2

Original Experiment October 5th, 2020

Last Updated March 3rd, 2022

Dr. Ashraf Suyyagh - All Rights Reserved

Revision History

Ver. 1.2 (March 2022)

Merged the first and third Experiments together

Removed discussion for hexadecimal and binary number formats section.

Removed the Integer Arithmetic section

Removed the time and date sections

Removed serial dates section

Removed the characters and strings sections

Removed the sections related to writing and loading excel sheets and text files using the command line and keep the GUI import tool yet moved it to the second experiment.

Ver. 1.1 (July 2021)

Corrected formatting and spelling mistakes

Added the *diary* and *type* commands.

Added discussion for hexadecimal and binary number formats.